1ª avaliação   ALUNO: Caio da silva Gouvea
Papel TEMPO: 1 Hora 30 minutos
computador: 1 HORA
4,5

```
import java.util.Arrays;

Public Class Prova {

        // Funções auxiliares

  Public static boolean contem(int[] v, int tam, int x
{
   for (int i =0; i < tam; i++ {
        if (v[i] == x) {
              return true;
        }
     }
     return false
   }
     Public static void inverter(int[] v, int ini
     int fim) {
          while (ini < fim) {
             int aux = v[ini];
             v[ini] = v[fim];
             v[fim] = aux;
             ini++;
             fim--;
        }
     }
// A) união

Public static uniao(int[] a, int tamA, int
[] b, int tamB, int[] u) {
                                        1,5
    int tamU = 0;
    for (int i = 0; i < tamA; i++) {
      if (!contem(u, tamU. a[i])) {
        u[tamU] = a[i];
        tamU++;
      }
    }
    for (int i = 0; i < tamB; i++) {
       if (!contem(u, tamU, b[i])) {
          u[tamU] = b[i];
          tamU++;
       }
    }
    retur tamU;
  }

// B) Insertion sort

Public static void ordena(int[] v, int n) {
    for (int i = 1; i < n; i++) {
    int chave = v[i];                 1,0
    int j = i - 1;
      while (j >= 0 && v[j] > chave) {
          v[j+1] = v[j];
          j--;
      }
      v[j+1] = chave;
    }
}
```

data ..../........./............
Ⓢ Ⓣ Ⓠ Ⓠ Ⓢ Ⓢ Ⓓ

```
// C) vetor sem repetição

  public static int gerarVetor(int[] v, int tamV,
  int[] vsr) {
                  int tamVSR = 0;
                  for (int i=0; i < tamV; i++){
                  if (!Contem(vsr, tamVSR, v[i]
                  )) {
                  vsr[tamVSR] = v[i];
                  tamVSR++;
                                          1,5
               }
           }
             return tamVSR;
       }

// D) rotação in place

  public static void rotacionar(int[] v, int tam int k)
{
 for (int i=0; i < k; i++){
 int Primeiro = v[0];
 for (int j = 0; j < tam; j++) {
        v[j] = v[j+1];
  }
  v[tam - 1] = Primeiro;
                                          0,5
 }
}
          // não trata k negativo caso não execute
       // guardou 1º elemento
       // deveria ser j < tam - 1
// em j = tam - 1 acessa v[tam] → ArrayIndexOutOfBounds
// recolocou no fim
// falta normalizar k (k % tam)
```